

EDUCATIONAL PROJECT

Playlist Downloader

A local web application for importing exported song lists and retrieving the matching audio

Technical Report

Flask • yt-dlp • SQLite • Vanilla JavaScript

Every figure in this report is a screenshot of the application running on a local machine. The measurements in Section 8 were taken from a single genuine execution on 2026-09-17.

MIT licensed • 2026-09-17

Abstract

Playlist Downloader is a small, self-hosted web application that closes the gap between a playlist someone has curated on a streaming service and a set of local audio files. The user exports a playlist to a plain TXT or CSV file, drops that file into the application, and the server searches YouTube for each entry, retrieves the best match with `yt-dlp`, converts it to MP3 or MP4, and returns the whole set as a single ZIP archive. A second entry point accepts YouTube video and playlist URLs directly.

The project is deliberately modest in scope: roughly three thousand lines across a Flask backend, a dependency-free frontend and a two-table SQLite log. It requires no user accounts, no API keys and no third-party service registration. This report describes its architecture, walks through the implementation of each subsystem, presents the results of a real execution, and sets out the limitations that follow from its design.

Scope note. This is an educational project. It demonstrates API integration, subprocess-driven media handling and progress reporting over HTTP. It is not intended to be deployed publicly, and using it to obtain copyrighted material without permission is a matter for the user, not the software. [Section 2](#) treats this properly.

Contents

Abstract	1
1 Introduction	4
1.1 Motivation	4
1.2 Goals	4
1.3 Non-goals	4
2 Legal and Ethical Scope	4
3 System Overview	5
3.1 Architecture	5
3.2 Technology choices	5
3.3 Size	6
4 Requirements	6
4.1 Functional	6
4.2 Non-functional	6
5 Design	6
5.1 Request flow	6
5.2 Data model	6
5.3 HTTP interface	8
5.4 Configuration	8
6 Implementation	8
6.1 Import and parsing	8
6.2 Search and retrieval	9
6.3 Progress reporting	9
6.4 Packaging and delivery	9
6.5 Administrative view	10
7 Running the Project	10
7.1 Installation	10
7.2 Launching	10
8 Results	12
8.1 Import	12
8.2 Download	12
8.3 What the archive contained	12
8.4 YouTube playlist inspection	13
8.5 Administrative record	13
9 Evaluation and Limitations	15
9.1 Matching is a single search query	15
9.2 Progress state is global	15
9.3 The request that works is the request that responds	15
9.4 Known defects	15
9.5 What works well	15
10 Future Work	16

11 Conclusion	16
A Repository Layout	17
B Reproducing the Figures	17

1 Introduction

1.1 Motivation

Streaming services are convenient until connectivity is not guaranteed. A curated playlist lives inside a provider's application and cannot easily be taken somewhere else: onto a plane, into a car with no data coverage, or onto a device that does not run the provider's client. The gap is narrow but real, and it is a good vehicle for a teaching project because closing it requires touching several unrelated technologies at once.

1.2 Goals

The project set out to satisfy five goals:

1. **No credentials.** The application must work without the user registering for an API key or signing into anything.
2. **Import from a neutral format.** Accept the plain text and CSV exports that playlist-export tools already produce, rather than binding the project to one provider's API.
3. **Visible progress.** A multi-minute operation must report what it is doing, item by item, including failures.
4. **Nothing left behind.** By default, files are streamed to the user's browser and not retained on the server.
5. **Local by construction.** The application should be pleasant to run on one machine and should not invite public deployment.

1.3 Non-goals

The project does not attempt audio fingerprinting or fuzzy matching beyond a single search query; it does not manage a library, tag files beyond their filename, or support incremental synchronisation. Multi-user operation is explicitly out of scope, and Section 9 explains why the current design could not support it without substantial change.

2 Legal and Ethical Scope

A report on a tool of this kind is incomplete without stating plainly what the tool does and does not license the user to do.

The application performs two operations that are individually unremarkable: it issues a search query to a public website, and it retrieves a media stream from a public URL. Neither operation circumvents access controls or digital rights management. What makes the combination sensitive is intent: the same mechanism that retrieves a Creative Commons recording retrieves a commercial one.

The position taken by this project is therefore:

- The software is distributed for study. Its value as a teaching artefact lies in the integration work — subprocess management, streaming responses, progress reporting — not in the media it fetches.
- Responsibility for what is retrieved rests entirely with the operator, who must comply with the terms of service of the platforms involved and with applicable copyright law.
- The application is not intended to be hosted publicly. It binds to a local port, ships with server-side storage disabled, and its administrative view is protected by a single shared password — adequate for a machine you control and wholly inadequate for anything else.

- The demonstration material accompanying this project uses public-domain classical recordings, so that nothing in the documentation depends on retrieving a work that is still in copyright.

3 System Overview

3.1 Architecture

The application is a single Flask process serving a single-page frontend. There is no build step, no bundler, no frontend framework and no message broker. The only external process is `yt-dlp`, invoked in-process as a Python library, which in turn calls `ffmpeg` to transcode audio.

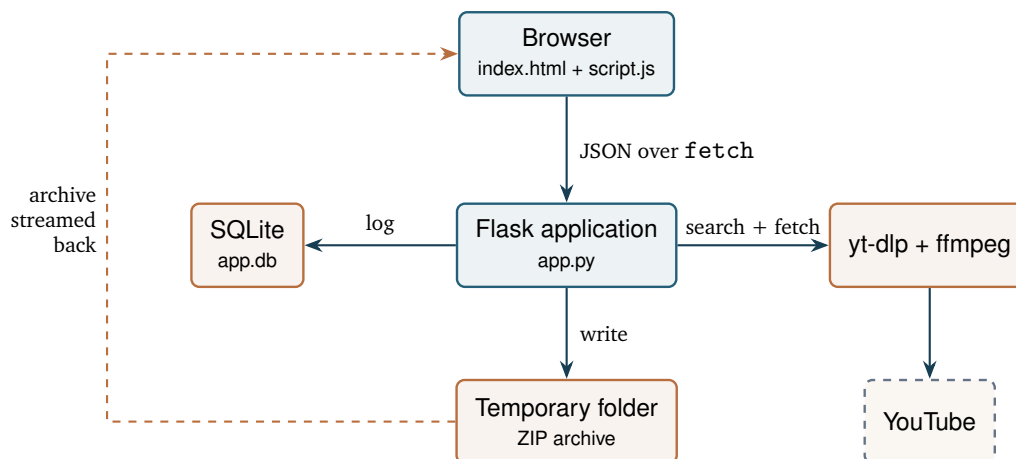


Figure 1: Component layout. Solid arrows are requests; the dashed arrow is the finished archive streamed back to the browser. Nothing is persisted outside the temporary folder unless server-side storage is explicitly enabled.

3.2 Technology choices

Layer	Choice	Rationale
Backend	Flask 3	One file, no ceremony; the routing surface is small enough that a larger framework would add cost without benefit.
Media	yt-dlp	Actively maintained, handles extraction and post-processing, exposes a Python API rather than only a CLI.
Transcoding	ffmpeg	Invoked by yt-dlp's post-processor to produce MP3 at a fixed 192 kbps.
Persistence	SQLite	Two tables of local activity; a server would be disproportionate.
Frontend	Vanilla JS + CSS	No build step means the project can be read top to bottom and run with one command.

Table 1: Technology choices and the reasoning behind each.

3.3 Size

File	Lines	Responsibility
app.py	754	Routes, search, download, archiving, admin
static/script.js	719	Parsing, rendering, polling, downloads
static/style.css	968	Presentation
templates/index.html	240	Main page structure
database.py	183	Schema and query helpers
templates/admin.html	177	Dashboard structure and its inline script
Total	3 041	

Table 2: Source size by file.

4 Requirements

4.1 Functional

- F1.** Import a playlist from a TXT or CSV file and list its tracks before any network activity occurs.
- F2.** Download every track in an imported playlist as MP3 or MP4.
- F3.** Download a single YouTube video by URL.
- F4.** Inspect a YouTube playlist by URL and download the whole set.
- F5.** Report progress during a run: current item, completed items, failed items with a reason.
- F6.** Pause and resume an in-flight run.
- F7.** Deliver results either as a browser download or, when explicitly enabled, to a server-side folder.
- F8.** Record activity and download history, and expose them through a password-protected dashboard.

4.2 Non-functional

- N1.** No account, API key or third-party registration.
- N2.** Run from a clean checkout with two commands.
- N3.** Default to retaining nothing on the server.
- N4.** Degrade honestly: a track that cannot be matched is reported, never silently dropped.

5 Design

5.1 Request flow

Figure 2 traces the playlist download, which is the most involved of the flows. The important structural detail is that the browser makes *two concurrent* kinds of request: one long-lived POST that performs the work and eventually returns the archive, and a one-second GET poll that reads a progress structure the POST is mutating as it goes.

5.2 Data model

The schema is deliberately minimal. There are no users, so there are no joins.

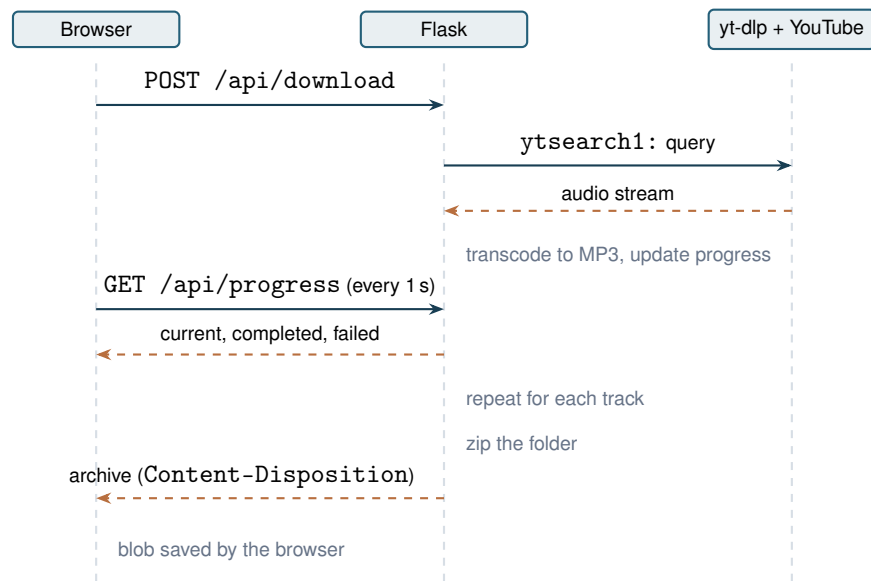


Figure 2: Playlist download. The progress poll runs alongside the long request that is doing the work, which is what makes per-track feedback possible without websockets.

Table	Column	Purpose
activity_logs	id	Primary key
	action	Machine-readable event name
	details	Human-readable description
	ip_address	Requesting address
	timestamp	Defaults to CURRENT_TIMESTAMP
download_history	id	Primary key
	download_type	playlist, youtube or youtube_playlist
	item_name	Track or video title
	playlist_url	Owning playlist, where applicable
	success	Boolean outcome
	error_message	Reason on failure
	ip_address	Requesting address
	timestamp	Defaults to CURRENT_TIMESTAMP

Table 3: SQLite schema. `database.py` calls `init_db()` at import time, so the file is created on first run with no migration step.

5.3 HTTP interface

Method	Path	Behaviour
GET	/	Main page
GET	/api/config	Reports whether server-side storage is enabled
POST	/api/download	Runs an imported playlist; returns JSON or the archive
GET	/api/progress	Current progress structure
POST	/api/stop	Requests a pause at the next track boundary
POST	/api/resume	Clears the pause flag
POST	/api/youtube/download	Single video by URL
POST	/api/youtube/playlist/info	Playlist metadata, without downloading
POST	/api/youtube/playlist/download	Whole YouTube playlist
GET	/downloads/<path>	Serves a stored file when server storage is on
GET	/admin	Dashboard (HTTP Basic)
GET	/api/admin/stats	Aggregate counters (HTTP Basic)
GET	/api/admin/activity	Recent activity (HTTP Basic)
GET	/api/admin/downloads	Download history (HTTP Basic)

Table 4: The complete route surface — fourteen endpoints, four of them behind the admin password.

5.4 Configuration

Configuration is read from the environment at import time, with a `.env` file loaded by `python-dotenv`.

Variable	Default	Effect
FLASK_PORT	5000	Listening port
DOWNLOAD_FOLDER	downloads	Server-side output directory
ALLOW_SERVER_STORAGE	false	Exposes the “Save to Server” option
ADMIN_USERNAME	admin	Dashboard user
ADMIN_PASSWORD	unset	Dashboard password; unset disables the dashboard entirely
SECRET_KEY	random	Flask session key

Table 5: Environment configuration. With `ADMIN_PASSWORD` unset, `is_admin_authorized()` returns `False` unconditionally, so the dashboard is closed rather than open by default.

6 Implementation

6.1 Import and parsing

Parsing happens entirely in the browser, in `parseImportedPlaylist()`. The file is never uploaded; the server only ever sees the resulting array of `{artist, name}` objects when the user chooses to download. This is worth emphasising because it means importing a playlist and inspecting it costs nothing and reveals nothing.

The parser handles three shapes without being told which it has been given. It first splits the opening line and looks for a header row; if one is present it maps columns by name, and otherwise it falls back to line-by-line parsing.

Listing 1: Header detection, from `static/script.js`.

```
const firstColumns = splitCsvLine(lines[0]).map((c) => c.toLowerCase());
const hasHeader = firstColumns.some((c) => /track|title|song|artist/.test(c));
```

Each individual line is normalised by `lineToTrack()`, which strips a leading track number, removes surrounding quotes, and then tries an “Artist - Title” split before falling back to CSV columns and finally to treating the whole line as a title with an unknown artist.

6.2 Search and retrieval

Matching is done with a single search query rather than a catalogue lookup:

Listing 2: Query construction, from `app.py`.

```
def search_youtube_video(song_name, artist):
    """Find best matching YouTube video"""
    query = f"{artist} {song_name} official audio".strip()
    return f"ytsearch1:{query}"
```

The `ytsearch1:` prefix instructs `yt-dlp` to run the search and take the first result, which folds search and retrieval into one call. It is a pragmatic choice with a real cost, discussed in Section 9.

Retrieval is a straightforward `yt-dlp` invocation whose options differ by requested format. Audio requests download the best available audio stream and hand it to the `FFmpegExtractAudio` post-processor at a fixed 192 kbps; video requests ask for the best MP4 video and M4A audio and merge them.

6.3 Progress reporting

Progress lives in a module-level dictionary, `download_progress`, which the download loop mutates as it advances and `/api/progress` serialises on demand. The frontend polls once a second and re-renders the panel:

Listing 3: The polling loop, from `static/script.js`.

```
function startProgressPolling() {
    stopProgressPolling();
    progressInterval = setInterval(updateProgress, 1000);
}
```

Polling stops when the reported status becomes completed, paused or error. Pausing is cooperative: `/api/stop` sets a `should_stop` flag, and the loop checks it at the top of each iteration, so a pause takes effect at the next track boundary rather than interrupting a transfer.

6.4 Packaging and delivery

When the request asked for a device download, the server writes into a `tempfile.mkdtemp()` directory, zips the files of the requested extension, and returns the archive with `send_from_directory(..., as_attachment=True)`. Cleanup is registered on the response rather than performed inline, so the temporary files survive exactly as long as the transfer:

Listing 4: Deferred cleanup, from `app.py`.

```
@response.call_on_close
def cleanup():
    try:
        shutil.rmtree(playlist_folder)
        os.remove(zip_path)
    except:
        pass
```

On the browser side, `downloadBlob()` materialises the response as an object URL, clicks a synthetic anchor and revokes the URL, which is what produces the ordinary browser save dialog seen in Section 8.

6.5 Administrative view

Every download attempt and every notable action is written to SQLite through `database.py`. The dashboard is a second template with an inline script that renders four counters and two tables. Access is guarded by a decorator that compares credentials with `hmac.compare_digest`, avoiding a timing side-channel on the password comparison:

Listing 5: Constant-time credential check, from `app.py`.

```
return (
    hmac.compare_digest(auth.username or '', ADMIN_USERNAME)
    and hmac.compare_digest(auth.password or '', ADMIN_PASSWORD)
)
```

7 Running the Project

7.1 Installation

Listing 6: From a clean checkout.

```
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env          # optional; every value has a default
```

`ffmpeg` must be present on the system path; `yt-dlp` shells out to it for the MP3 conversion.

7.2 Launching

Listing 7: Starting the application.

```
python app.py
```

The console output below was captured from the run that produced the figures in this report:

Listing 8: Real console output on startup.

```
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5099
* Running on http://192.168.x.x:5099
Press CTRL+C to quit
```

The development server binds to `0.0.0.0`, which makes it reachable from other machines on the same network. That is convenient on a trusted LAN and inappropriate anywhere else; see Section 9.

Opening <http://localhost:5000> gives the page shown in Figure 3.

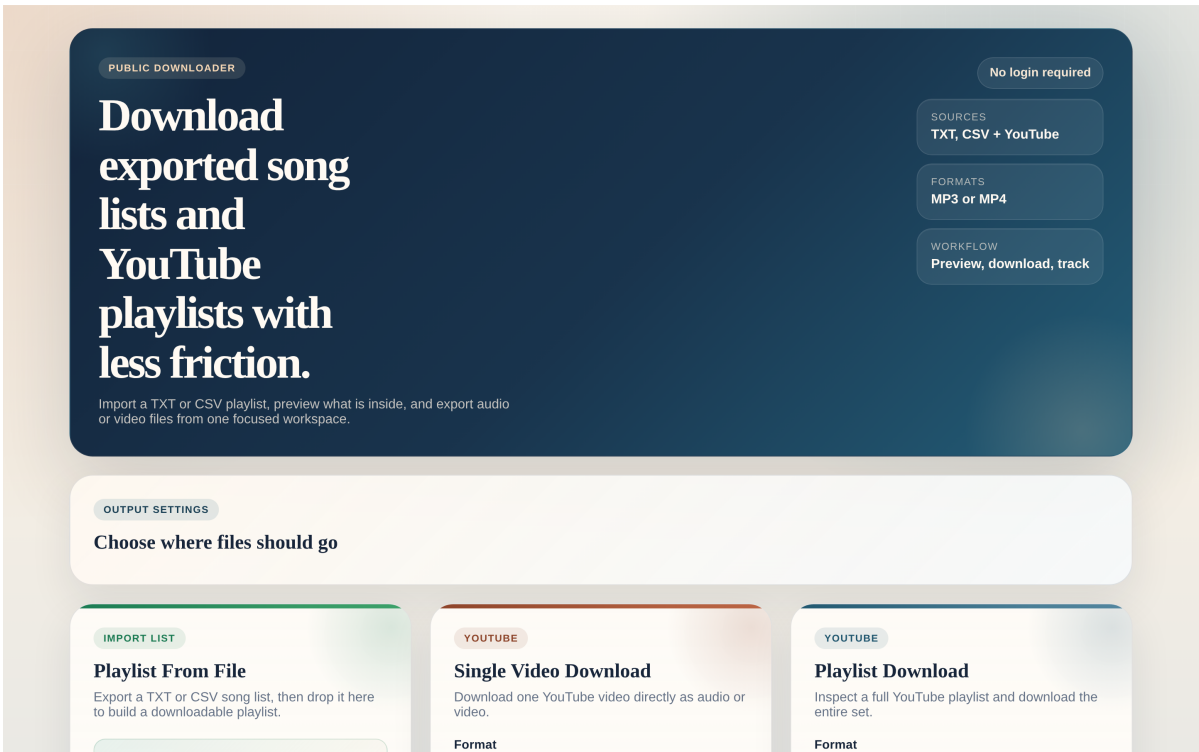


Figure 3: The application immediately after launch. The three entry points are below the fold; the “Output Settings” panel is empty here because server-side storage is disabled by default.

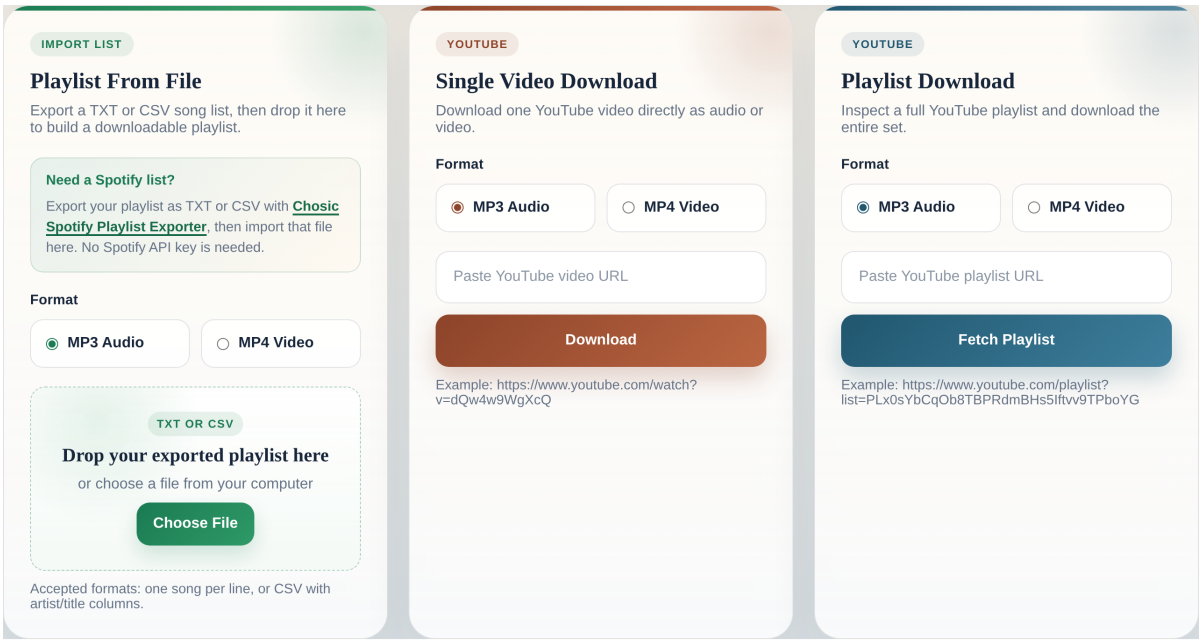


Figure 4: The three entry points: file import, a single YouTube video, and a YouTube playlist. Each carries its own MP3/MP4 selector.

8 Results

This section reports a single genuine execution performed on 2026-09-17. The playlist was a four-track file of public-domain classical recordings; no step was simulated.

8.1 Import

Dropping the file produced the listing in Figure 5 essentially instantly, since no network request is involved.



Figure 5: The parsed playlist. The title is taken from the filename and the 4 tracks are listed with artist attribution before any download begins.

8.2 Download

The run completed 4 of 4 tracks with 0 failures in 40.5 s, an average of 10.1 s per track. That average includes a deliberate one-second pause between tracks, so the true cost per track is closer to nine seconds: a search, a transfer of a few megabytes, and an MP3 conversion.

8.3 What the archive contained

The delivered ZIP was 19.8 MB and held 13:40 of audio. The table below was produced by reading the archive back with `ffprobe` after the run, not from the application's own reporting.

File	Duration	Size	Bitrate
Claude Debussy - Clair de Lune.mp3	5:11	7.5 MB	192 kbps
Edvard Grieg - In the Hall of the Mountain King.mp3	2:46	4.0 MB	192 kbps
Ludwig van Beethoven - Fur Elise.mp3	2:53	4.2 MB	192 kbps
Scott Joplin - Maple Leaf Rag.mp3	2:50	4.1 MB	192 kbps
Total	13:40	19.8 MB	

Table 6: Verified contents of the archive produced by the run in Figure 6. The constant 192 kbps reflects the fixed `preferredquality` in the post-processor configuration.

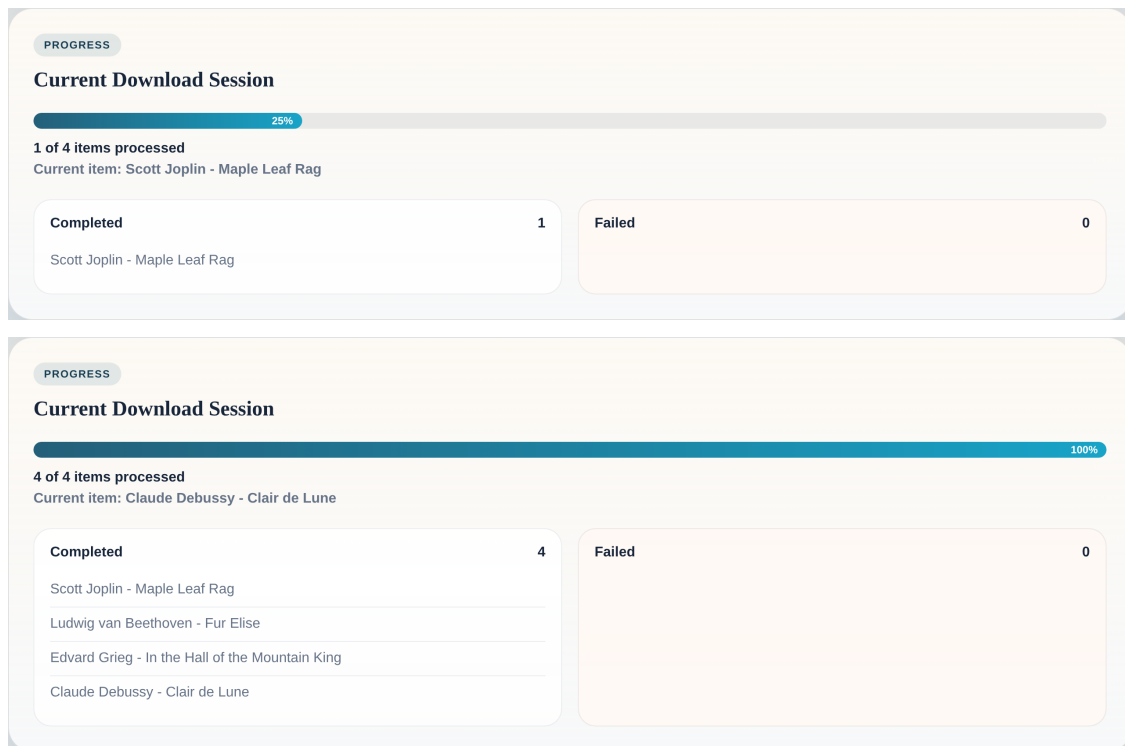


Figure 6: Progress in flight (top) and on completion (bottom). The panel names the item currently being processed and separates completed from failed work.

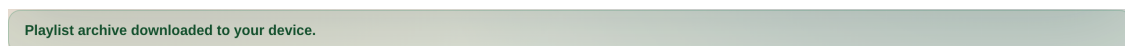


Figure 7: The confirmation shown once the archive has been handed to the browser.

8.4 YouTube playlist inspection

Pasting a playlist URL and pressing *Fetch Playlist* resolved “Blender Releases” to 19 videos without downloading anything, using *yt-dlp*’s flat extraction mode.

8.5 Administrative record

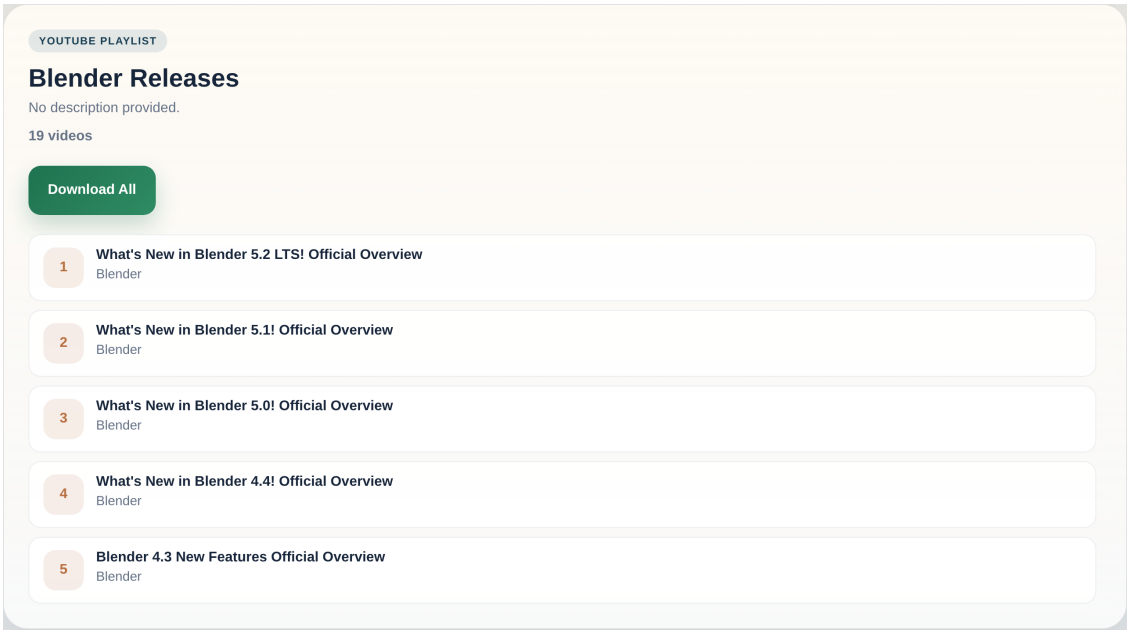


Figure 8: A real YouTube playlist resolved to its video list. The channel column reads “Unknown” throughout — a defect discussed in Section 9.

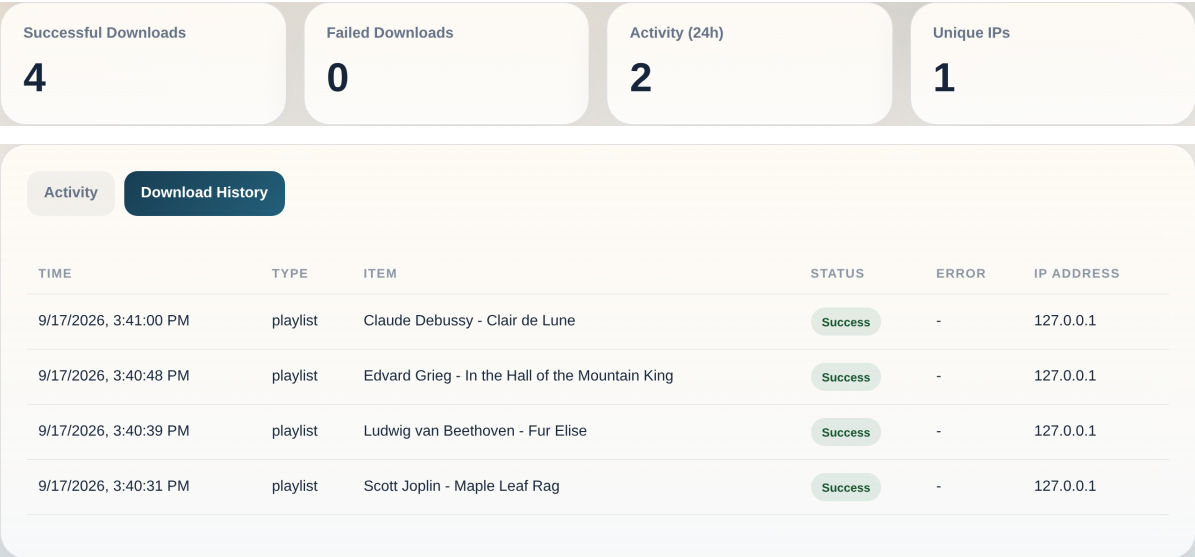


Figure 9: The dashboard after the run: aggregate counters (top) and the per-track history (bottom). Every row corresponds to one track from Figure 6.

9 Evaluation and Limitations

The project meets its functional requirements, but several design decisions have consequences worth stating precisely.

9.1 Matching is a single search query

`ytsearch1`: takes whatever YouTube ranks first. For distinctive titles this is reliable; for widely compiled material it is not. During preparation of the demonstration material, the query for one well-known piano piece resolved to a sixty-five-minute compilation instead of the single recording — a 94 MB file where 5 MB was expected. The application had no way to notice: the download succeeded, so it was reported as a success.

A duration sanity check against the expected track length, or presenting the candidate for confirmation, would close most of this gap.

9.2 Progress state is global

`download_progress` is a module-level dictionary. Two simultaneous runs — two browser tabs, or two people on the same LAN — would overwrite each other’s progress and produce an archive containing both sets of files. This is a direct consequence of the single-user assumption and is the first thing that would have to change to support more than one operator.

9.3 The request that works is the request that responds

A playlist download holds one HTTP request open for its entire duration. For four tracks that is forty seconds; for a two-hundred-track playlist it would be well beyond any reverse proxy’s default timeout. A job queue with a task identifier would be the conventional remedy.

9.4 Known defects

Area	Observation	Severity
Playlist metadata	<code>entry.get('uploader')</code> is not populated by flat extraction, so every channel renders as “Unknown” (Figure 8). <code>entry.get('channel')</code> carries the value.	Cosmetic
Output settings	When server storage is disabled the toggle is hidden but its containing card is not, leaving an empty panel (Figure 3).	Cosmetic
Dependency floor	<code>requirements.txt</code> permits <code>yt-dlp>=2023.0.0</code> . Releases more than a few months old are rejected by YouTube with HTTP 403 and every track fails.	Functional
Error handling	The cleanup handler catches bare <code>except:</code> , which would also swallow an interrupt.	Minor
Binding	<code>app.run(host='0.0.0.0', debug=True)</code> exposes the debugger to the local network.	Security

Table 7: Defects identified during this evaluation, with the severity each carries for a single-user local deployment.

9.5 What works well

The import path is the strongest part of the design. Parsing in the browser means a user can inspect a playlist with no network traffic at all, the server never receives the file, and the format-guessing logic is easy to test in isolation. The progress panel’s separation of completed

and failed work is also worth keeping: it makes partial success legible, which is the normal outcome for a search-based matcher.

10 Future Work

1. **Confirm the match.** Present the candidate video and its duration before downloading, or reject results whose length differs substantially from the expected track.
2. **Move work off the request.** A task queue with an identifier would remove both the global-state and long-request problems at once.
3. **Stream progress.** Server-sent events would replace the one-second poll with something proportionate.
4. **Tag the output.** Writing ID3 metadata from the imported playlist would make the archive useful to a music library rather than a folder of filenames.
5. **Raise the dependency floor** and surface a clear message when `yt-dlp` is too old, rather than reporting every track as failed.

11 Conclusion

Playlist Downloader does a narrow job and does it visibly. In roughly three thousand lines it demonstrates browser-side file parsing, subprocess-driven media retrieval, progress reporting over a polled endpoint, streaming file delivery, and a small audited log — without requiring a single credential from the user.

Its weaknesses follow directly from its strengths. The decision to match tracks with one search query is what removes the need for API keys, and it is also what makes matching occasionally wrong. The decision to keep progress in a module global is what makes the implementation short, and it is also what confines the application to one user. For a project whose purpose is to be read and understood, those are defensible trades, provided they are stated rather than hidden.

The measurements in Section 8 come from one real execution: 4 tracks, 40.5 s, 19.8 MB delivered and verified. The application does what this report says it does.

References

- [1] Pallets Projects. *Flask Documentation*. <https://flask.palletsprojects.com/>
- [2] yt-dlp contributors. *yt-dlp*. <https://github.com/yt-dlp/yt-dlp>
- [3] FFmpeg team. *FFmpeg Documentation*. <https://ffmpeg.org/documentation.html>
- [4] SQLite Consortium. *SQLite Documentation*. <https://www.sqlite.org/docs.html>
- [5] Chosic. *Spotify Playlist Exporter*. <https://www.chosic.com/spotify-playlist-exporter/>

A Repository Layout

Listing 9: Files that matter, with generated output omitted.

app.py	Flask application: routes, search, download, archiving
database.py	SQLite schema and query helpers
requirements.txt	Python dependencies
start.sh / start.bat	Convenience launchers
.env.example	Documented configuration template
templates/	
index.html	Main page
admin.html	Dashboard
static/	
script.js	Parsing, rendering, polling, downloads
style.css	Presentation
docs/latex/	This report, the slides and their figures
demo_assets/	Demo video and the pipeline that produces it

B Reproducing the Figures

Every figure in Sections 7 and 8 is a screenshot of the application running, captured automatically. To regenerate them together with the measurements:

```
cd docs/latex
python capture_screens.py    # launches the app, performs a real run
python make_facts.py        # rewrites facts.tex from the measurements
make                        # builds report.pdf and slides.pdf
```

`capture_screens.py` starts the application from a temporary working directory, so regenerating the figures never touches an existing `app.db` or `downloads/` folder.